UNIVERSIDADE FEDERAL DA BAHIA
INSTITUTO DE COMPUTAÇÃO

GUILHERME PIMENTEL SANTOS BARBOSA GOMES
ARTHUR PIMENTEL SANTOS BARBOSA GOMES

VETCARE: IMPLEMENTAÇÃO DO SISTEMA DE GESTÃO PARA CLÍNICA VETERINÁRIA

Etapa 2

SALVADOR
2026

GUILHERME PIMENTEL SANTOS BARBOSA GOMES
ARTHUR PIMENTEL SANTOS BARBOSA GOMES

VETCARE: IMPLEMENTAÇÃO DO SISTEMA DE GESTÃO PARA CLÍNICA VETERINÁRIA

Etapa 2

Trabalho apresentado à disciplina Engenharia de Software 1, do Instituto de Computação da Universidade Federal da Bahia, como requisito parcial de avaliação (Trabalho II).

Professora: Dra. Larissa Barbosa Leôncio Pinheiro.

SALVADOR
2026

SUMÁRIO

1 INTRODUÇÃO **3**
2 REQUISITOS IMPLEMENTADOS NA ETAPA **4**
2.1 Critérios de seleção 4
2.2 Requisitos selecionados 4
2.3 Requisitos fora desta etapa 5
3 ARQUITETURA IMPLEMENTADA **6**
3.1 Visão geral 6
3.2 Correspondência entre a Etapa 1 e o código 6
3.3 Táticas arquiteturais 7
4 REGISTRO DE DECISÕES ARQUITETURAIS **9**
4.1 ADR-010: linguagem e framework 9
4.2 ADR-011: banco de dados 10
4.3 ADR-012: hash de senha 10
4.4 ADR-013: barramento de eventos e Outbox 11
4.5 ADR-014: cache da agenda 11
4.6 ADR-015: porta de leitura do Cadastro 12
5 IMPLEMENTAÇÃO **13**
5.1 Tecnologias 13
5.2 Estrutura do repositório 13
5.3 Regra de conflito de horário 13
5.4 Controle de acesso por perfil 14
5.5 Interface de programação (API) 14
6 SISTEMA EM FUNCIONAMENTO **16**
7 TESTES E VERIFICAÇÃO **20**
8 EXECUÇÃO E REPOSITÓRIO **21**
9 CONSIDERAÇÕES FINAIS **22**
REFERÊNCIAS **23**

1 INTRODUÇÃO

Este documento apresenta a primeira etapa de implementação do VetCare, sistema de gestão para clínicas veterinárias especificado no Documento de Requisitos da disciplina (PINHEIRO, 2026). O objetivo da etapa foi implementar aproximadamente 30% dos requisitos funcionais, em um sistema que funcione de ponta a ponta e que siga a arquitetura definida no Trabalho I (GOMES; GOMES, 2026): um monolito modular com arquitetura hexagonal (portas e adaptadores) dentro de cada módulo.

Como a Etapa 1 não fixou a linguagem de programação, a implementação foi feita em Python. A escolha do framework e os pontos em que a implementação se afastou da arquitetura original estão registrados como ADRs (*Architecture Decision Records*) na seção 4.

Como resultado, foram implementados 6 dos 17 requisitos funcionais (35%), todos de prioridade alta; 4 módulos da arquitetura estão em funcionamento (Identidade e Acesso, Cadastro, Agenda e Auditoria); o Portal Web apresenta telas conforme o perfil do usuário; e a solução conta com 54 testes automáticos, incluindo testes que verificam a regra de dependência entre os módulos. O código-fonte está disponível em repositório público no GitHub (seção 8).

2 REQUISITOS IMPLEMENTADOS NA ETAPA

2.1 Critérios de seleção

Foram selecionados 6 dos 17 requisitos funcionais, o que corresponde a 35% do total, valor próximo aos 30% solicitados. A seleção seguiu três critérios:

- prioridade: somente requisitos classificados como de prioridade alta;
- dependência: os demais requisitos dependem destes, pois não há atendimento, prontuário ou pagamento sem usuário autenticado, tutor, animal, veterinário e consulta;
- fatia vertical completa: esta etapa entrega um fluxo útil de ponta a ponta, em que o administrador cria os usuários e a recepção cadastra tutor e animal e agenda a consulta com um veterinário.

2.2 Requisitos selecionados

O Quadro 1 relaciona os requisitos implementados, o módulo da Etapa 1 a que pertencem e os critérios de aceitação atendidos.

Quadro 1 – Requisitos funcionais implementados na Etapa 2

REQ	Requisito	Módulo	Critérios de aceitação implementados
REQ 01	Manter Usuário	Identidade e Acesso	Inclusão, alteração, exclusão e consulta de usuários com perfil (Administrador, Recepcionista, Veterinário). E-mail único; senha com 8 ou mais caracteres, letras e números, armazenada apenas como *hash*; o sistema mantém sempre um administrador ativo; nenhum usuário exclui ou desativa a si mesmo. Busca e paginação.
REQ 02	Autenticar Usuário	Identidade e Acesso	Login por e-mail e senha com *token* assinado (validade de 8 h). Mensagem genérica para e-mail ou senha incorretos; bloqueio de 15 min após 5 tentativas falhas; usuário desativado perde o acesso mesmo com *token* válido; menu e telas exibidos conforme o perfil.
REQ 04	Manter Tutor	Cadastro	Cadastro, alteração, consulta e exclusão. CPF validado pelos dígitos verificadores e único; telefone com DDD; e-mail e endereço opcionais. Não exclui tutor que possua animais.
REQ 05	Manter Animal	Cadastro	Cadastro, alteração, consulta e exclusão, sempre associado a um tutor existente, com nome, espécie, raça, sexo e data de nascimento (não futura). Filtro por tutor. Não exclui animal com consultas registradas.
REQ 06	Manter Veterinário	Cadastro	Cadastro e gestão de veterinários: nome, CRMV único (normalizado para `CRMV-UF 12345`), especialidade, contato e situação ativo/inativo. Veterinário inativo não recebe consultas.
REQ 07	Agendar Consulta	Agenda	Agendamento relacionando animal, tutor, veterinário, data e horário. Somente no futuro, de segunda a sábado, das 8 h às 18 h, na grade de 30 min. Impede sobreposição para o mesmo veterinário e para o mesmo animal, com restrição única no banco contra dupla marcação simultânea. Exibe os horários livres e lista as consultas do dia.

Fonte: Elaborado pelos autores (2026).

2.3 Requisitos fora desta etapa

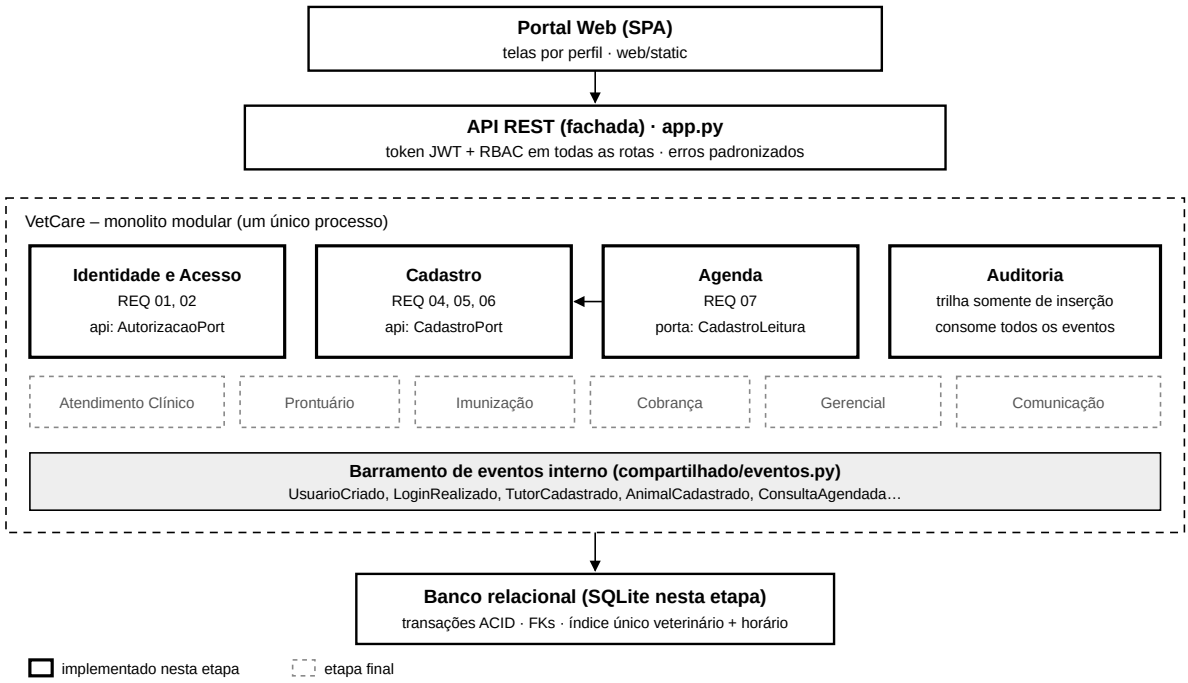
Ficaram para a etapa final os requisitos REQ 03 (Recuperar Senha), REQ 08 (Cancelar Consulta), REQ 09 (Consultar Agenda por dia, semana ou profissional), REQ 10 (Registrar Atendimento), REQ 11 (Manter Prontuário), REQ 12 (Registrar Vacinação), REQ 13 (Emitir Prescrição), REQ 14 (Registrar Pagamento), REQ 15 (Dashboard Financeiro), REQ 16 (Notificar Tutor) e REQ 17 (Gerar Relatórios Gerenciais). A listagem simples das consultas do dia foi incluída como apoio ao REQ 07, para conferência do agendamento; as visões por semana e por profissional do REQ 09 serão implementadas na etapa final.

3 ARQUITETURA IMPLEMENTADA

3.1 Visão geral

O estilo definido na Etapa 1 foi mantido: uma única aplicação implantável, dividida em módulos por domínio. Cada módulo possui a camada `dominio` (entidades, regras e portas, sem dependência de banco de dados ou da web), a camada `aplicacao` (serviços que coordenam os casos de uso) e a camada `adaptadores` (repositório SQLite e controlador HTTP), e expõe apenas o arquivo `api.py` como interface pública. A Figura 1 mostra os componentes implementados nesta etapa.

Figura 1 – Componentes da Etapa 1 implementados na Etapa 2



Fonte: Elaborado pelos autores (2026).

3.2 Correspondência entre a Etapa 1 e o código

O Quadro 2 indica onde cada elemento da arquitetura proposta na Etapa 1 foi implementado no código-fonte.

Quadro 2 – Elementos da arquitetura e sua localização no código

Elemento da Etapa 1	**Localização no código**
API REST (fachada)	`vetcare/app.py`: converte as rotas dos módulos para o Starlette, valida o *token* e a permissão em todas as rotas e transforma exceções de domínio em erros JSON padronizados.
Identidade e Acesso	`identidade/`: controladores em `adaptadores/http.py`; `AutenticacaoService` e `UsuarioService`; política RBAC em `dominio/politica_acesso.py`; adaptadores de *hash*, *token* e repositório; `AutorizacaoPort` em `api.py`.
Agenda	`agenda/`: `AgendamentoService`, `DisponibilidadeService` (regra pura de conflito de horário), repositório com índice único, evento `ConsultaAgendada` e porta consumida `CadastroLeitura`.
Tutores e Animais; Equipe Clínica	`cadastro/`: entidades `Tutor`, `Animal` e `Veterinario`; serviços; repositórios; interface pública `CadastroPort`.
Auditoria	`auditoria/modulo.py`: inscrita em todos os eventos; tabela protegida contra alteração e exclusão por gatilhos; consulta restrita ao Administrador.
Compartilhado e infraestrutura	Eventos, exceções e tipos comuns; conexão com o banco, configuração externalizada e contrato HTTP neutro.
Portal Web	`web/static/`: SPA com menu lateral e cabeçalho com o usuário logado; agenda do dia como tela inicial da recepção.

Fonte: Elaborado pelos autores (2026).

3.3 Táticas arquiteturais

O Quadro 3 apresenta a situação, nesta etapa, das táticas arquiteturais definidas na Etapa 1.

Quadro 3 – Situação das táticas arquiteturais na Etapa 2

Característica	**Tática**	**Situação**	**Como foi feita**
Segurança	Hash de senha com sal	Feita	scrypt com sal de 16 bytes e parâmetros da OWASP (ADR-012).
	RBAC na fachada e na regra de negócio	Feita	Permissão declarada em cada rota; política centralizada; nega por padrão.
	Bloqueio após falhas e expiração de sessão	Feita	5 tentativas resultam em 15 min de bloqueio; *token* expira em 8 h; usuário revalidado a cada requisição.
	Validação de entrada e consultas parametrizadas	Feita	Validação no domínio; SQL sempre parametrizado; HTML escapado; cabeçalhos de segurança (CSP, X-Frame-Options).
Integridade	Transações e restrições no banco	Feita	Transações ACID, chaves estrangeiras com `RESTRICT` e índice único parcial veterinário + horário ativo.
	Idempotência e prontuário versionado	Etapa final	Pertencem aos módulos Cobrança e Prontuário.

Manutenibilidade	Módulos por domínio, interfaces e eventos	Feita	Cada módulo expõe apenas `api.py`; integração por eventos.
	Regra de dependência verificada na CI	Feita	`tests/test_arquitetura.py` executado no GitHub Actions.
	Configuração externalizada	Feita	Variáveis de ambiente.
Disponibilidade	Verificação de saúde	Feita	Rota `/api/saude` testa o banco.
	Degradação graciosa; backup	Parcial	Falha em consumidor de evento não interrompe a operação; backup após a migração para PostgreSQL (ADR-011).
Desempenho	Índices e paginação	Feita	Índices da agenda; listagens paginadas (máximo de 100).
	Cache da agenda do dia	Adiada	Desnecessário no volume medido (ADR-014).
Auditabilidade	Trilha somente de inserção a partir de eventos	Feita	Gatilhos no banco bloqueiam alterações; acesso restrito ao Administrador.
Usabilidade	Navegação consistente e erros padronizados	Feita	Mesmo layout em todas as telas; mensagens de erro em português indicando o campo.
Testabilidade	Domínio isolado e portas substituíveis	Feita	Regras de agenda e login testadas sem banco, com relógio e repositórios simulados.

Fonte: Elaborado pelos autores (2026).

4 REGISTRO DE DECISÕES ARQUITETURAIS

As ADR-001 a ADR-009 da Etapa 1 continuam válidas, exceto nos pontos alterados a seguir. A numeração prossegue a partir da ADR-010. O Quadro 4 resume as novas decisões, detalhadas nas subseções seguintes. O mesmo conteúdo está no repositório, no arquivo `docs/ADRs-Etapa2.md`.

Quadro 4 – Resumo das ADRs da Etapa 2

ADR	**Título**	**Relação com a Etapa 1**
ADR-010	Linguagem e framework: Python 3, Starlette e Uvicorn	Complementa a ADR-002
ADR-011	SQLite nesta etapa; PostgreSQL mantido para produção	Desvio parcial da ADR-004
ADR-012	Hash de senha com scrypt	Desvio da tática de hash (ADR-003)
ADR-013	Barramento síncrono; Outbox adiada para o REQ 16	Desvio de cronograma da ADR-007
ADR-014	Cache da agenda do dia adiado	Desvio parcial da ADR-009
ADR-015	Agenda declara a própria porta de leitura do Cadastro	Refinamento da ADR-005

Fonte: Elaborado pelos autores (2026).

4.1 ADR-010: linguagem e framework

Quadro 5 – ADR-010: Python 3, Starlette e Uvicorn

Status	Aceita.
Contexto	A Etapa 1 definiu o estilo arquitetural (ADR-002), mas não a linguagem nem o framework. A equipe é pequena e o sistema precisa ser fácil de instalar e executar na máquina do avaliador.
Decisão	Python 3.10 ou superior, com Starlette (servidor ASGI enxuto) e Uvicorn. O Portal Web é uma SPA em HTML, CSS e JavaScript sem framework, servida pela própria aplicação. A fachada (`app.py`) traduz um contrato neutro (`Rota`, `Requisicao`, `Resposta`) para o Starlette, de modo que os módulos não importam o framework.
Alternativas	Django, que impõe ORM e organização em camadas globais, contrária à arquitetura hexagonal por módulo; FastAPI, descartado para não levar o modelo do Pydantic ao domínio e reduzir dependências; SPA com React ou Vue, que exigiria processo de *build* sem ganho para seis telas.
Consequências	Positivas: três dependências de execução; domínio em Python puro; instalação com um único comando. Negativas: ausência de documentação OpenAPI automática (endpoints documentados no README); a SPA sem framework exige disciplina para não crescer desorganizada.

Fonte: Elaborado pelos autores (2026).

4.2 ADR-011: banco de dados

Quadro 6 – ADR-011: SQLite nesta etapa, PostgreSQL em produção

Status	Aceita; altera parcialmente a ADR-004.
Contexto	A ADR-004 escolheu PostgreSQL. Para a avaliação, o sistema precisa ser executado sem que o avaliador instale e configure um servidor de banco de dados.
Decisão	Usar SQLite (biblioteca padrão do Python) nesta etapa, mantendo as garantias que motivaram a ADR-004: transações ACID, chaves estrangeiras com `ON DELETE RESTRICT`, restrição única parcial `(veterinario_id, inicio) WHERE status='AGENDADA'` e auditoria protegida por gatilhos. O acesso ao banco fica apenas nos adaptadores de repositório, atrás de portas; migrar para PostgreSQL consiste em escrever novos adaptadores, sem alterar domínio nem serviços.
Alternativas	PostgreSQL via Docker desde já, mais fiel à ADR-004, porém com mais um pré-requisito para executar o projeto.
Consequências	Positivas: o sistema é executado com um único comando. Negativas: o SQLite serializa escritas (suficiente para dezenas de usuários); o backup com restauração a um instante (ADR-006) só será possível após a migração, prevista antes da implantação.

Fonte: Elaborado pelos autores (2026).

4.3 ADR-012: hash de senha

Quadro 7 – ADR-012: hash de senha com scrypt

Status	Aceita; altera a tática "senha com hash e sal (Argon2 ou bcrypt)".
Contexto	Argon2 e bcrypt exigem bibliotecas com código nativo. O objetivo da tática, de que um vazamento do banco não exponha as senhas, deve ser mantido sem dependências nativas.
Decisão	Usar scrypt (`hashlib.scrypt`) com sal aleatório de 16 bytes por senha e parâmetros $N = 2^{17}$, $r = 8$, $p = 1$, mínimo recomendado pela OWASP quando o Argon2id não está disponível (OWASP FOUNDATION, [s. d.]). Os parâmetros ficam gravados junto do hash e a comparação usa tempo constante. O adaptador implementa a porta `GeradorDeHash`; trocar para Argon2 significa trocar um adaptador.
Alternativas	Argon2id, preferível, mas com dependência nativa; PBKDF2, disponível na biblioteca padrão, porém sem uso intensivo de memória, o que facilita ataques com GPU.
Consequências	Positivas: função com uso intensivo de memória, sem dependência adicional. Negativas: cada login leva cerca de 0,3 s e usa 128 MiB durante a verificação; os parâmetros precisam ser revistos com o tempo.

Fonte: Elaborado pelos autores (2026).

4.4 ADR-013: barramento de eventos e Outbox

Quadro 8 – ADR-013: barramento síncrono e Outbox adiada

Status	Aceita; adia parte da ADR-007.
Contexto	A ADR-007 prevê o padrão Outbox para notificar o tutor com retentativa. Nesta etapa ainda não há notificações (REQ 16) e o único consumidor de eventos é a Auditoria.
Decisão	Implementar o barramento interno com entrega síncrona, dentro da mesma transação do fato de origem; uma falha no consumidor não interrompe a operação de quem publicou. A Agenda já publica `ConsultaAgendada`. Quando o módulo Comunicação for criado, ele gravará esses eventos na Outbox e um processo separado fará o envio, conforme a ADR-007.
Alternativas	Criar a Outbox nesta etapa, sem nenhum consumidor.
Consequências	Positivas: auditoria gravada de forma atômica com a operação; nenhum código sem uso. Negativas: um consumidor lento atrasaria a operação, o que é aceitável enquanto o único consumidor é uma inserção local.

Fonte: Elaborado pelos autores (2026).

4.5 ADR-014: cache da agenda

Quadro 9 – ADR-014: cache da agenda do dia adiado

Status	Aceita; adia parte da ADR-009.
Contexto	A ADR-009 previa cache de curta duração para a agenda do dia. A equipe mediu o desempenho com cerca de 12.850 consultas (dez meses de agenda cheia para cinco veterinários) e 300 tutores: a listagem de um dia com 50 consultas levou 1,4 ms (p95) e o cálculo de horários livres, 0,1 ms (p95), medidos na camada de aplicação. A meta da Etapa 1 é de 2 s para 95% das consultas.
Decisão	Não implementar cache nesta etapa. Manter índices em `(veterinario_id, inicio)`, `(animal_id, inicio)` e `(inicio)`, além da paginação. Reavaliar com nova medição quando o REQ 09 for implementado.
Alternativas	Implementar o cache nesta etapa, como previsto.
Consequências	Positivas: sem risco de exibir agenda desatualizada e sem lógica de invalidação. Negativas: com o crescimento da carga, o cache pode voltar a ser necessário, e poderá ser incluído como adaptador da porta de leitura.

Fonte: Elaborado pelos autores (2026).

4.6 ADR-015: porta de leitura do Cadastro

Quadro 10 – ADR-015: Agenda declara a própria porta de leitura do Cadastro

Status	Aceita; refina a ADR-005.
Contexto	Na Etapa 1, a Agenda consome a `CadastroPort`. Se o núcleo da Agenda importasse a classe do Cadastro, o domínio da Agenda passaria a depender de outro módulo.
Decisão	O núcleo da Agenda declara a porta `CadastroLeitura` (com `animal(id)` e `veterinario(id)`). O Cadastro publica a `CadastroPort` em `cadastro/api.py` e o ponto de composição liga uma à outra. Testes de arquitetura verificam que o domínio não importa infraestrutura, que os módulos só usam o `api.py` uns dos outros, que as dependências seguem a tabela da Etapa 1 e que não há ciclos.
Alternativas	Importar a `CadastroPort` diretamente no serviço da Agenda.
Consequências	Positivas: a Agenda é testada com um Cadastro simulado; a dependência Agenda → Cadastro existe em tempo de execução, como no diagrama, sem acoplamento de código. Negativas: uma interface a mais para manter.

Fonte: Elaborado pelos autores (2026).

5 IMPLEMENTAÇÃO

5.1 Tecnologias

O Quadro 11 apresenta as tecnologias adotadas em cada camada da solução.

Quadro 11 – Tecnologias utilizadas

Camada	**Tecnologia**
Linguagem	Python 3.10 ou superior
Servidor web e API	Starlette e Uvicorn (ASGI)
Banco de dados	SQLite (módulo `sqlite3`), modo WAL, chaves estrangeiras ativas
Autenticação	*Token* JWT (PyJWT, HS256) e hash scrypt (`hashlib`)
Interface	SPA em HTML, CSS e JavaScript, sem framework
Testes e integração contínua	pytest e GitHub Actions

Fonte: Elaborado pelos autores (2026).

5.2 Estrutura do repositório

O código está organizado em pacotes por domínio, conforme o diagrama de pacotes da Etapa 1. O Quadro 12 mostra a estrutura de diretórios.

Quadro 12 – Estrutura de diretórios do repositório

```
vetcare/
├── app.py              ponto de composição e fachada REST
├── __main__.py         python -m vetcare (inicia o servidor)
├── dados_exemplo.py    python -m vetcare.dados_exemplo
├── compartilhado/      eventos de domínio, exceções e tipos comuns
├── infraestrutura/     banco, configuração e contrato HTTP neutro
├── identidade/         REQ 01, 02   dominio/ aplicacao/ adaptadores/ api.py
├── cadastro/           REQ 04-06    dominio/ aplicacao/ adaptadores/ api.py
├── agenda/             REQ 07       dominio/ aplicacao/ adaptadores/ api.py
├── auditoria/          trilha somente de inserção
└── web/static/         Portal Web (index.html, app.css, app.js)
tests/                  54 testes: domínio, serviços, API e arquitetura
docs/ADRs-Etapa2.md     ADR-010 a ADR-015
.github/workflows/      integração contínua (pytest)
```

Fonte: Elaborado pelos autores (2026).

5.3 Regra de conflito de horário

A regra de agendamento fica no domínio, sem dependência de banco ou da web, e por isso é testada isoladamente. O serviço de aplicação chama essa regra e grava a consulta na mesma transação; o índice único no banco é a última barreira contra a dupla marcação. O Quadro 13 apresenta o trecho principal.

Quadro 13 – Trecho da classe DisponibilidadeService

```
class DisponibilidadeService:
    def validar(self, inicio, agora, consultas_do_veterinario,
consultas_do_animal):
        fim = inicio + timedelta(minutes=self.duracao_min)
        if inicio <= agora:
            raise DadosInvalidos("Não é possível agendar consultas no
passado.")
        ...  # grade de 30 min e horário de funcionamento
        if any(c.ativa and c.sobrepoe(inicio, fim) for c in
consultas_do_veterinario):
            raise Conflito("O veterinário já possui consulta neste horário.")
        if any(c.ativa and c.sobrepoe(inicio, fim) for c in
consultas_do_animal):
            raise Conflito("Este animal já possui consulta neste horário.")
```

Fonte: Elaborado pelos autores (2026).

5.4 Controle de acesso por perfil

As permissões ficam centralizadas em uma única tabela no módulo Identidade e Acesso. Cada rota declara a permissão exigida e a fachada a verifica antes de chamar o módulo, conforme o Quadro 14.

Quadro 14 – Política RBAC centralizada

```
PERMISSOES = {
    "usuarios:gerenciar": {ADMINISTRADOR},
    "tutores:ler":        {ADMINISTRADOR, RECEPCIONISTA, VETERINARIO},
    "tutores:escrever":   {ADMINISTRADOR, RECEPCIONISTA},
    "consultas:agendar":  {ADMINISTRADOR, RECEPCIONISTA},
    "auditoria:ler":      {ADMINISTRADOR},
    ...
}
Rota("POST", "/api/consultas", agendar, "consultas:agendar")
```

Fonte: Elaborado pelos autores (2026).

5.5 Interface de programação (API)

O Quadro 15 lista os endpoints da API REST, com a permissão exigida e o requisito atendido.

Quadro 15 – Endpoints da API REST

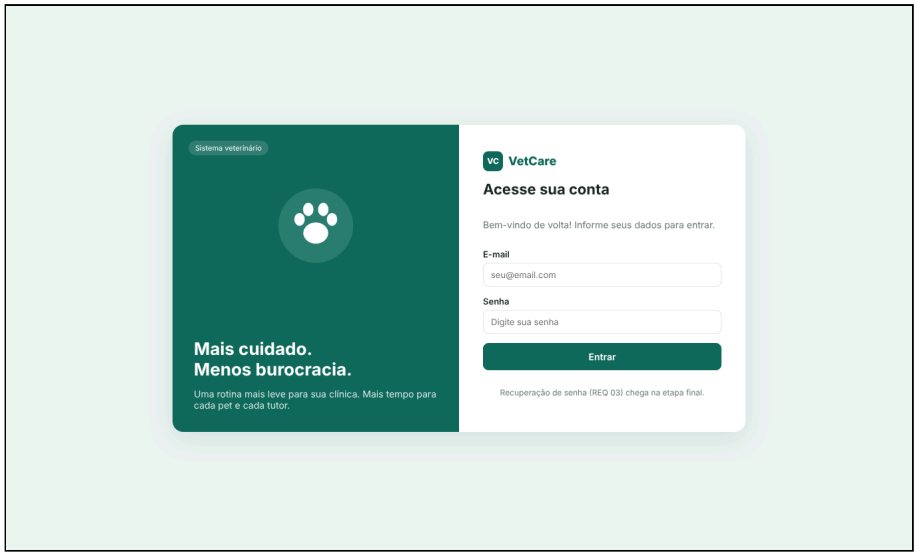
Método	**Rota**	**Permissão**	**REQ**
POST	`/api/auth/login`	Pública	02
GET	`/api/auth/eu`	Autenticado	02
GET, POST, PUT, DELETE	`/api/usuarios[/{id}]`	Administrador	01
GET, POST, PUT, DELETE	`/api/tutores[/{id}]`	Leitura: todos; escrita: Adm., Recepção	04
GET, POST, PUT, DELETE	`/api/animais[/{id}]`	Leitura: todos; escrita: Adm., Recepção	05
GET, POST, PUT, DELETE	`/api/veterinarios[/{id}]`	Leitura: todos; escrita: Administrador	06
POST	`/api/consultas`	Adm., Recepção	07
GET	`/api/consultas?data=`	Todos	07
GET	`/api/agenda/horarios-livres`	Todos	07
GET	`/api/auditoria`	Administrador	–
GET	`/api/saude`	Pública	–

Fonte: Elaborado pelos autores (2026).

6 SISTEMA EM FUNCIONAMENTO

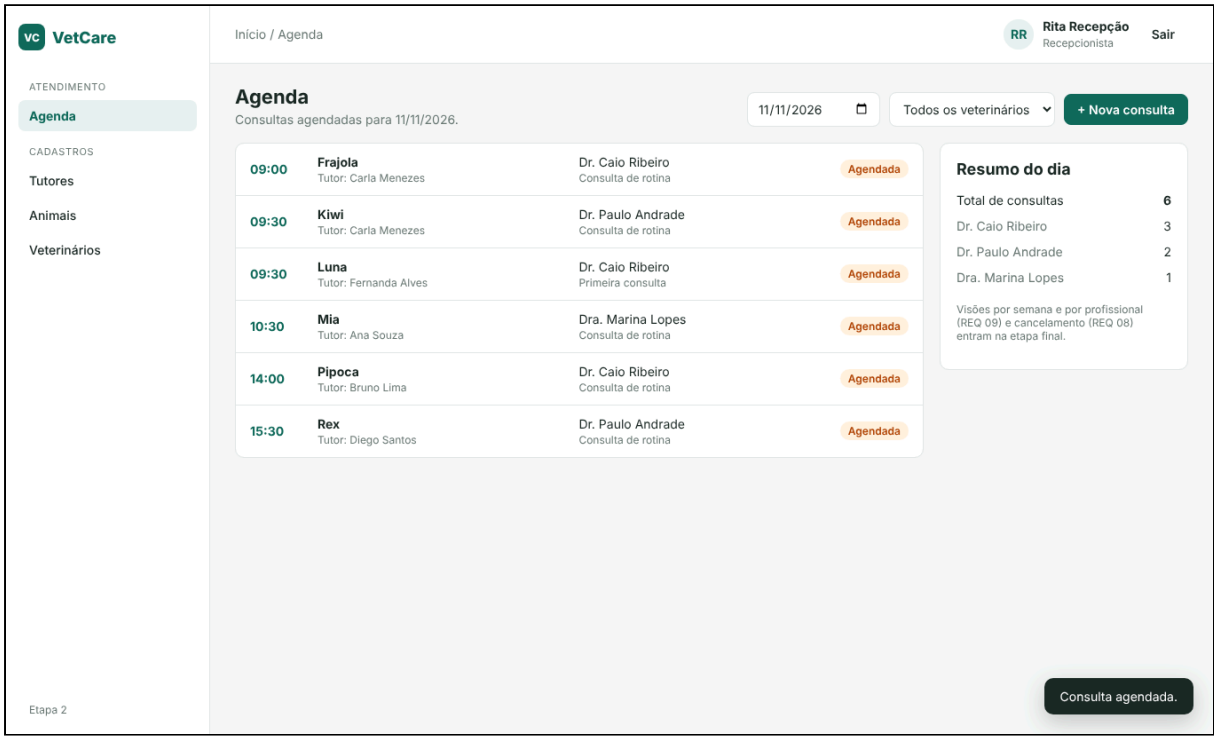
As telas seguem os protótipos da Etapa 1: menu lateral, cabeçalho com o usuário logado e a agenda do dia como tela inicial da recepção. O menu e os botões de ação aparecem conforme o perfil. As Figuras 2 a 9 foram capturadas com o sistema em execução e os dados de demonstração.

Figura 2 – Tela de login (REQ 02)



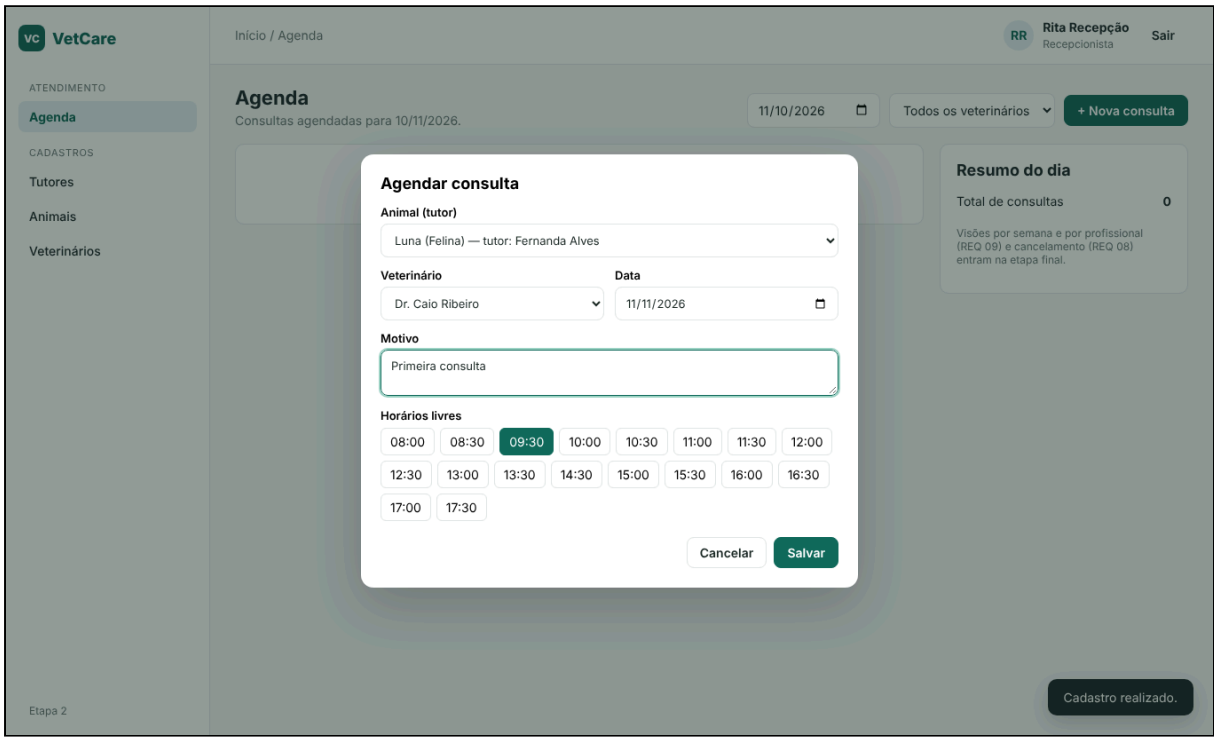
Fonte: Elaborado pelos autores (2026).

Figura 3 – Agenda do dia, perfil Recepcionista (REQ 07)



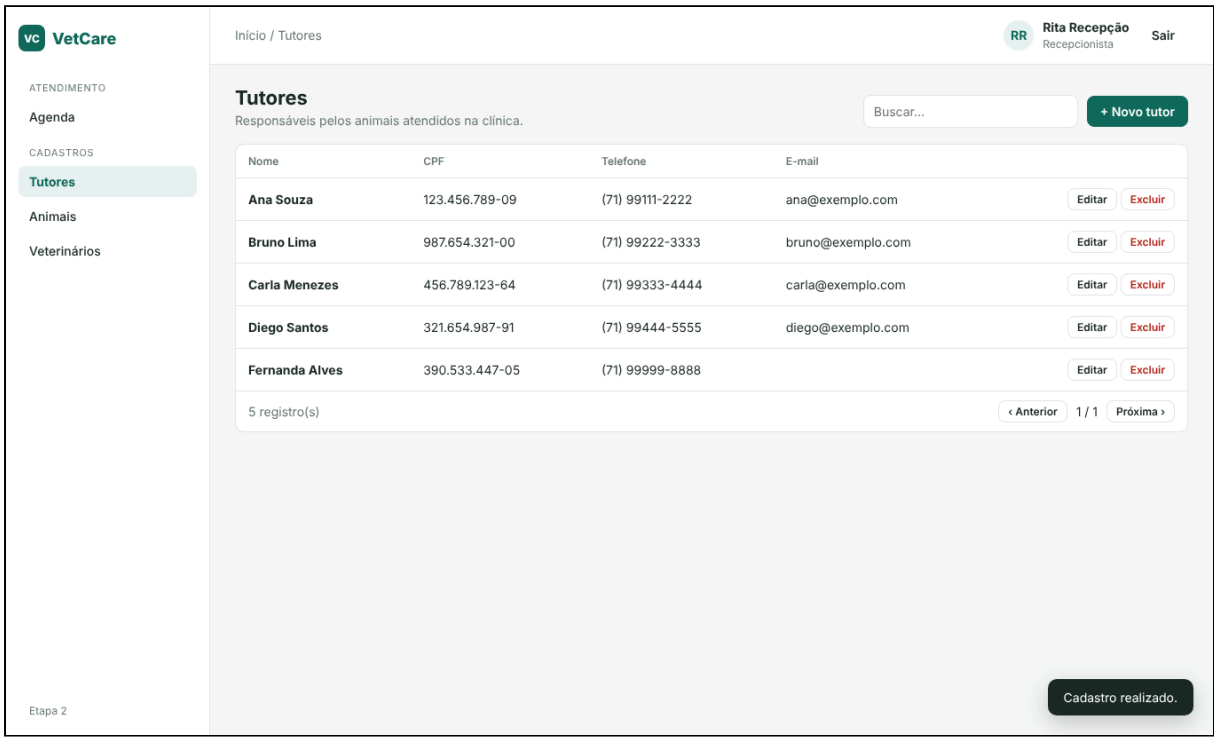
Fonte: Elaborado pelos autores (2026).

Figura 4 – Agendamento com horários livres calculados pelo sistema (REQ 07)



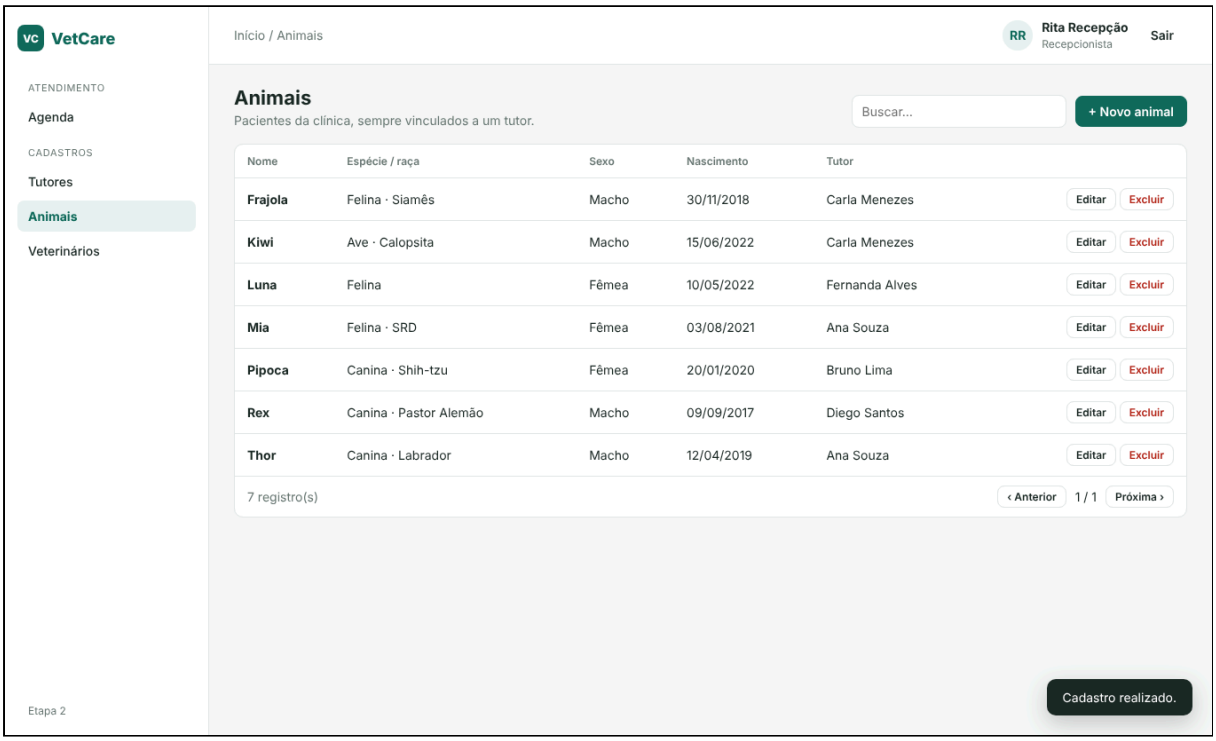
Fonte: Elaborado pelos autores (2026).

Figura 5 – Cadastro de tutores, com busca e paginação (REQ 04)



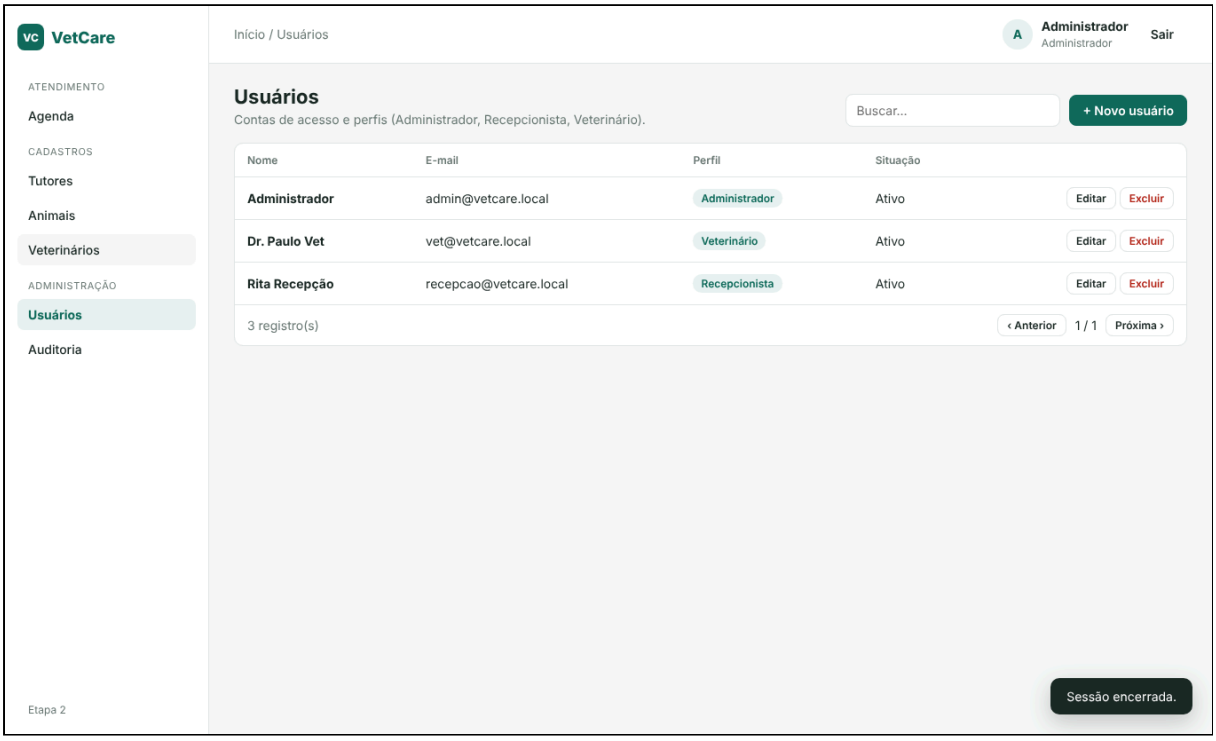
Fonte: Elaborado pelos autores (2026).

Figura 6 – Cadastro de animais vinculados ao tutor (REQ 05)



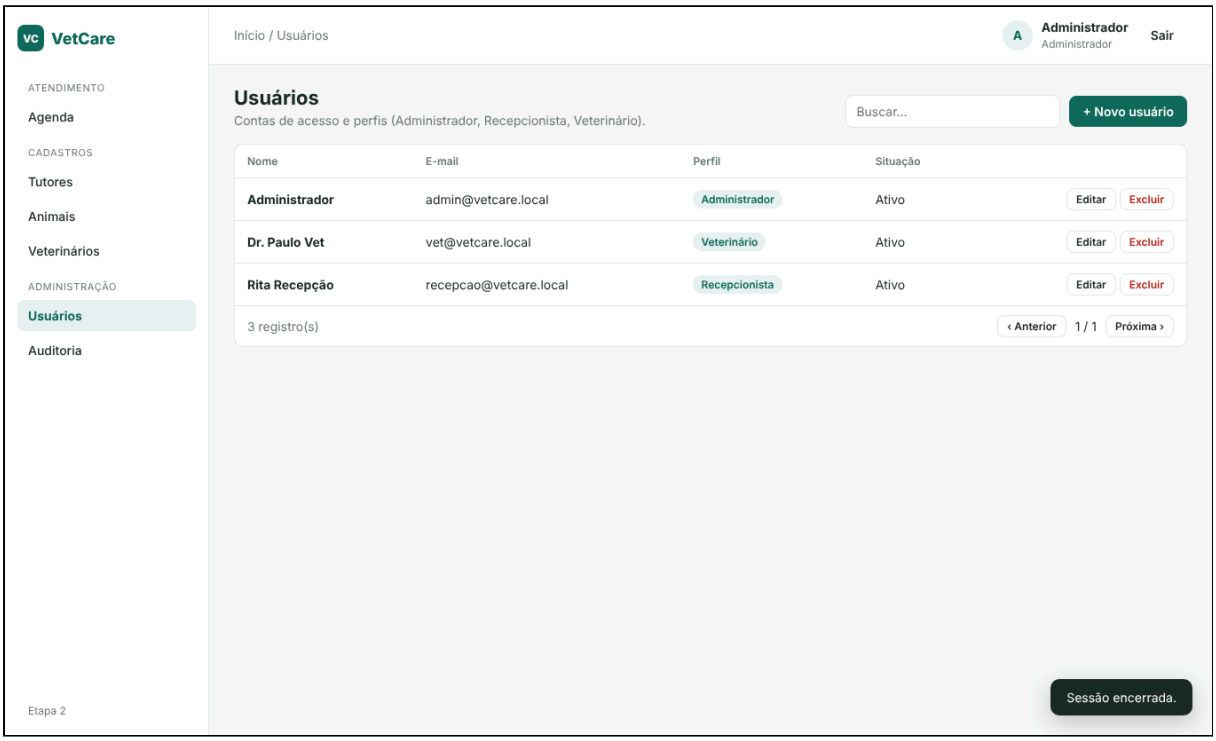
Fonte: Elaborado pelos autores (2026).

Figura 7 – Cadastro de veterinários, perfil Administrador (REQ 06)



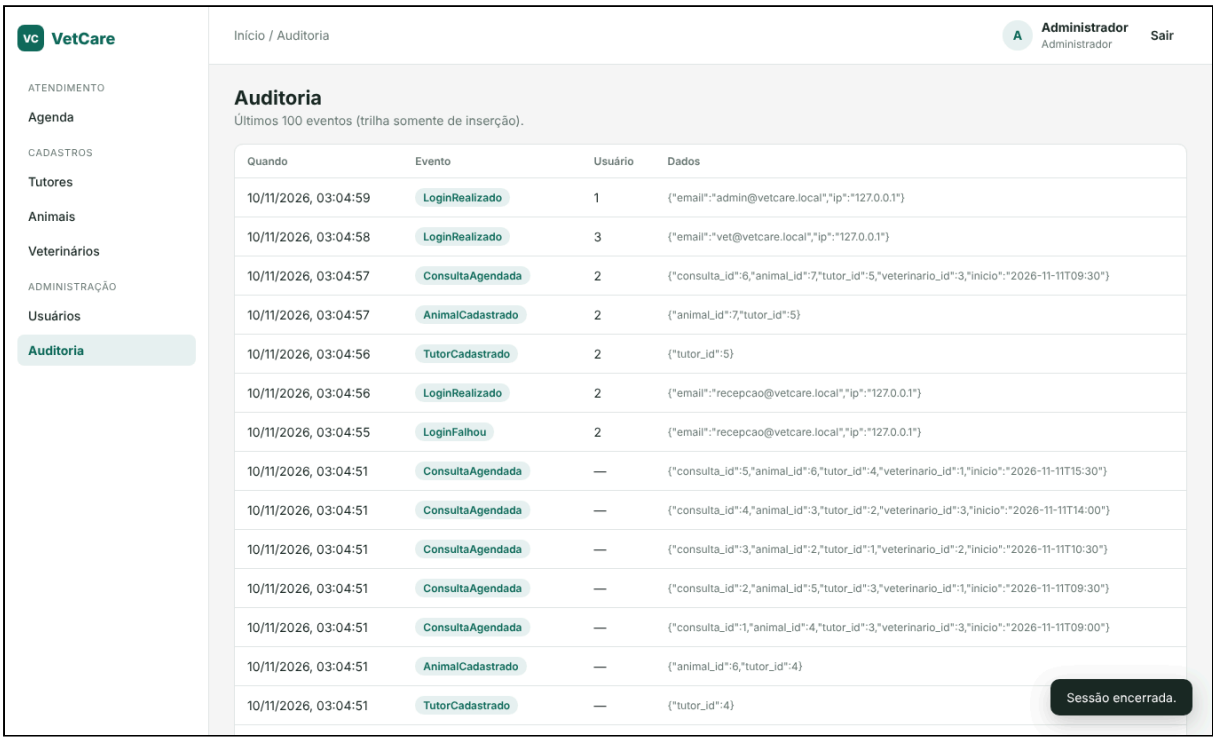
Fonte: Elaborado pelos autores (2026).

Figura 8 – Gestão de usuários e perfis, restrita ao Administrador (REQ 01)



Fonte: Elaborado pelos autores (2026).

Figura 9 – Trilha de auditoria alimentada por eventos



Fonte: Elaborado pelos autores (2026).

7 TESTES E VERIFICAÇÃO

A suíte possui 54 testes automáticos, todos aprovados, executados em cerca de 1 segundo com o pytest. O Quadro 16 apresenta a distribuição dos testes.

Quadro 16 – Distribuição dos testes automáticos

Arquivo	**Testes**	**O que verifica**
`test_identidade.py`	14	REQ 01 e 02 com objetos simulados: política de senha, e-mail único, último administrador, login, mensagem genérica, bloqueio e liberação após 15 min, usuário desativado, *token* adulterado ou expirado, hash e RBAC.
`test_cadastro.py`	16	REQ 04 a 06: CPF válido e inválido, normalização do CRMV, data de nascimento futura, CPF e CRMV duplicados, animal sem tutor e exclusão bloqueada por dependentes.
`test_agenda.py`	15	REQ 07: horário no passado, fora do expediente, domingo, grade de 30 min, conflito do veterinário e do animal, horários livres; restrição única do banco; quatro agendamentos simultâneos no mesmo horário resultam em exatamente uma consulta.
`test_api.py`	6	Fluxo pela API: erro 401 sem *token*, 403 por perfil, fluxo completo até o agendamento, 409 em conflito, formato padronizado de erro e paginação.
`test_arquitetura.py`	3	Regras do diagrama de pacotes: domínio sem infraestrutura, acesso entre módulos apenas por `api.py` e conforme a tabela de dependências, ausência de ciclos.

Fonte: Elaborado pelos autores (2026).

Os testes de arquitetura foram validados com a introdução proposital de uma violação, em que o domínio da Agenda passou a importar `sqlite3` e uma classe interna do Cadastro: os dois testes correspondentes falharam e apontaram o arquivo e a importação responsáveis.

Além da suíte automática, a interface foi testada no navegador com os três perfis: login com senha incorreta e correta, cadastro de tutor (incluindo o erro de CPF duplicado), cadastro de animal, agendamento pela escolha de horário livre e conferência dos menus por perfil e da trilha de auditoria.

8 EXECUÇÃO E REPOSITÓRIO

O código-fonte está disponível no repositório público `https://github.com/GuilhermePSBG/vetcare`. O arquivo `README.md` contém as instruções completas. O sistema exige Python 3.10 ou superior e é executado com os comandos do Quadro 17.

Quadro 17 – Comandos para executar o sistema

```
python -m venv .venv
.venv\Scripts\activate              (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python -m vetcare.dados_exemplo     (opcional: dados de demonstração)
python -m vetcare                   (acesso em http://127.0.0.1:8000)
pytest                              (executa os 54 testes)
```

Fonte: Elaborado pelos autores (2026).

O Quadro 18 apresenta os usuários disponíveis para teste.

Quadro 18 – Usuários de demonstração

Perfil	**E-mail**	**Senha**
Administrador (criado na primeira execução)	`admin@vetcare.local`	`Admin@123`
Recepcionista (dados de demonstração)	`recepcao@vetcare.local`	`Senha@123`
Veterinário (dados de demonstração)	`vet@vetcare.local`	`Senha@123`

Fonte: Elaborado pelos autores (2026).

9 CONSIDERAÇÕES FINAIS

A Etapa 2 entregou 35% dos requisitos funcionais do VetCare em um sistema executável de ponta a ponta, mantendo o estilo arquitetural definido na Etapa 1. Os desvios necessários foram registrados nas ADR-010 a ADR-015, e a regra de dependência entre módulos passou a ser verificada automaticamente, o que reduz o risco de degradação da arquitetura nas próximas etapas.

A etapa final do projeto deverá implementar os requisitos restantes, REQ 03 e REQ 08 a REQ 17: recuperação de senha com *token* de uso único, cancelamento e consulta da agenda por semana e por profissional, o módulo clínico (atendimento e prontuário versionado, somente com acréscimos), imunização, prescrição, cobrança com chave de idempotência, notificações com o padrão Outbox e relatórios gerenciais. Antes da implantação, estão previstas a migração para PostgreSQL, a rotina de backup e a publicação com HTTPS.

REFERÊNCIAS

GOMES, Guilherme Pimentel Santos Barbosa; GOMES, Arthur Pimentel Santos Barbosa. **VetCare**: documento de arquitetura do sistema. 2026. Trabalho I (Engenharia de Software 1) – Instituto de Computação, Universidade Federal da Bahia, Salvador, 2026.

OWASP FOUNDATION. **Password storage cheat sheet**. [*S. l.*]: OWASP, [*s. d.*]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 9 nov. 2026.

PINHEIRO, Larissa Barbosa Leôncio. **Documento de requisitos**: VetCare – sistema de gestão para clínica veterinária. Versão 01.00. Salvador: Instituto de Computação, Universidade Federal da Bahia, 2026.